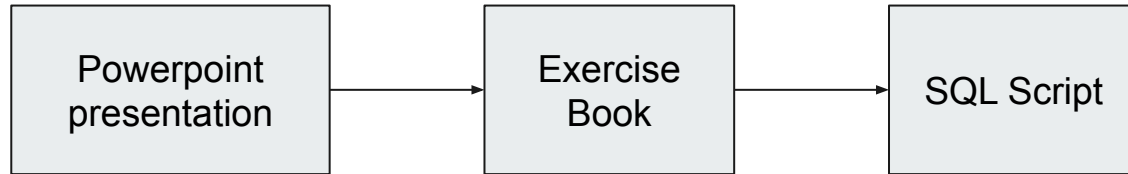




Data Aggregation and Database - Database Pathway



Architecture of the course and files





Course 1

INTRODUCTION TO DATABASES



Course 1

Introduction to relational databases, context of use, and use cases

Entity-Relationship(ER) model, use of primary key and foreign key

Tutorial : Transformation of a text to a ER model

Transformation of a ER model to a relational model



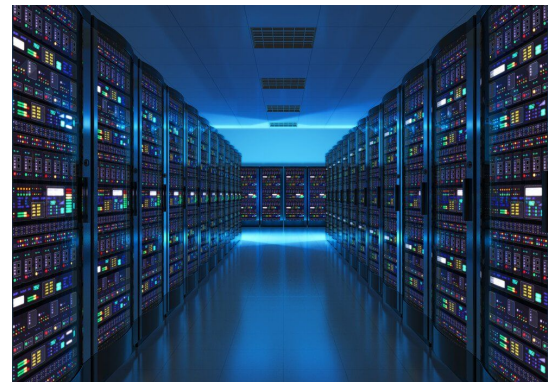
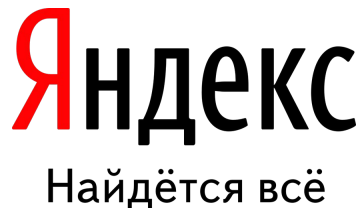
Introduction

Databases are used to store, organize and analyze data.

They designate a collection of information organized in order to facilitate the consultation of data, their management and their updating.

Databases presentation

Databases are now omnipresent in the business world but also in everyone's daily life.





History of database creation

The term **database** was born in 1964 to designate a collection of information shared by different users of a military information system.

1970s: Creation of Network Databases

Set of files linked by pointers

Navigation query language



History of database creation

1980s: Advent of Relational Databases

Relationships between data sets

Query language by logical assertion

1990s: Decision-making orientation (Data mining, OLAP)

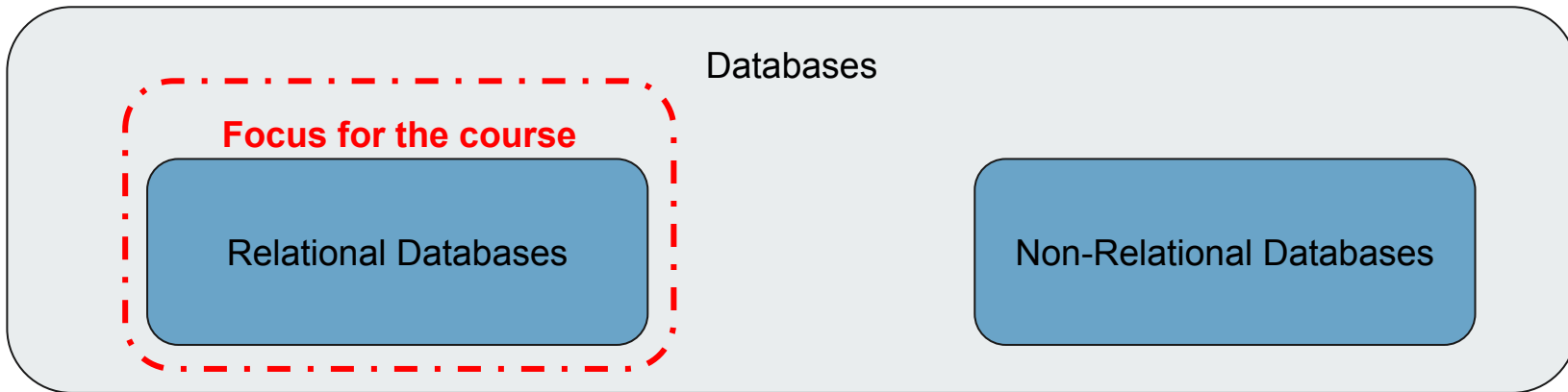
2000s: Advent of the Web

2010s - 2020s: Advent of the Cloud for databases



Introduction to relational databases

Today there are two main types of databases: Relational Databases and Non-Relational Databases.





Database issues

“Why use multiple linked databases instead of one big database with all the information inside?”



Introduction to relational databases

The main problem with relational databases comes in particular from the redundancy of information in a table. Let's take an example:

ID	Name	Surname	Class representative	Club1	Club2
1001245	Jean	Charles	Dufour	Poker	Finance
1001246	Dufour	Antoine	Dufour	Photo	Sport
1001247	Dupont	Marie	Dufour	Poker	Sport



Introduction to relational databases

When adding and deleting information from a database, there may be anomalies.

Insertion anomaly

Deletion anomaly

Modification anomaly



Introduction to relational databases

Insertion: If we add a student, he will necessarily have to join 2 clubs.

ID	Nom	Prénom	Class representative	Club1	Club2
1001245	Jean	Charles	Dufour	Poker	Finance
1001246	Dufour	Antoine	Dufour	Photo	Sport
1001247	Dupont	Marie	Dufour	Poker	Sport
1001248	Azevedo	Pierre	Dufour	?	?

Introduction to relational databases

Deletion: If the last student in a club is deleted, the club is automatically deleted.

ID	Nom	Prénom	Class representative	Club1	Club2
1001245	Jean	Charles	Dufour	Poker	Finance
1001246	Dufour	Antoine	Dufour	Photo	Sport
1001247	Dupont	Marie	Dufour	Poker	Sport

The Photo Club
Disappears



Introduction to relational databases

Modification: If a property associated with a class is changed, for example the delegate

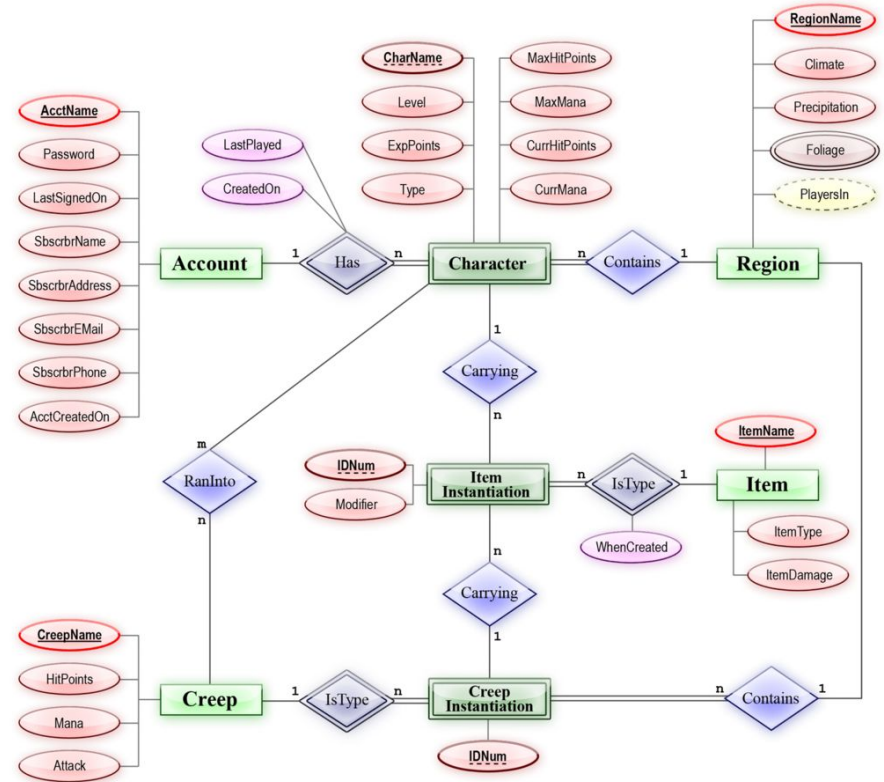
ID	Nom	Prénom	Class representative	Club1	Club2
1001245	Jean	Charles	Dufour	Poker	Finance
1001246	Dufour	Antoine	Dufour	Photo	Sport
1001247	Dupont	Marie	Dufour	Poker	Sport

Change
delegate name

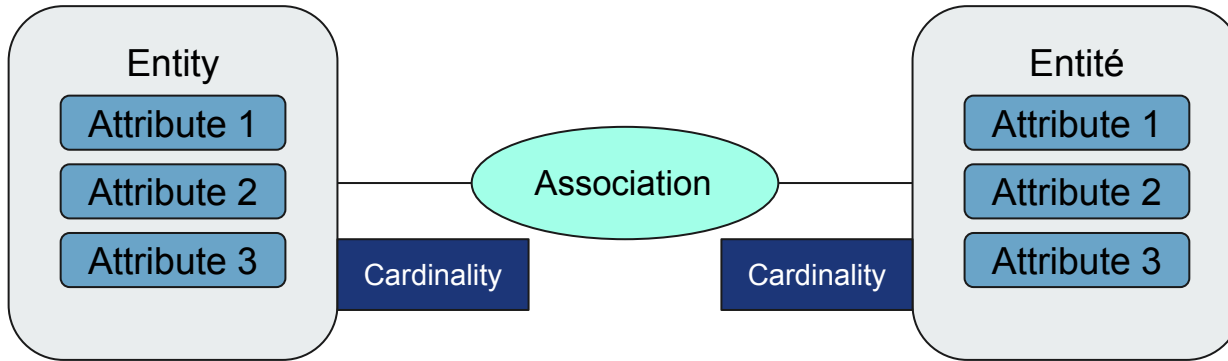
Solution

To overcome the anomaly issues, we have broken down the databases into smaller elements: Entity and Association. The resulting model is called:

The Entity-Relationship Model (or ER model)



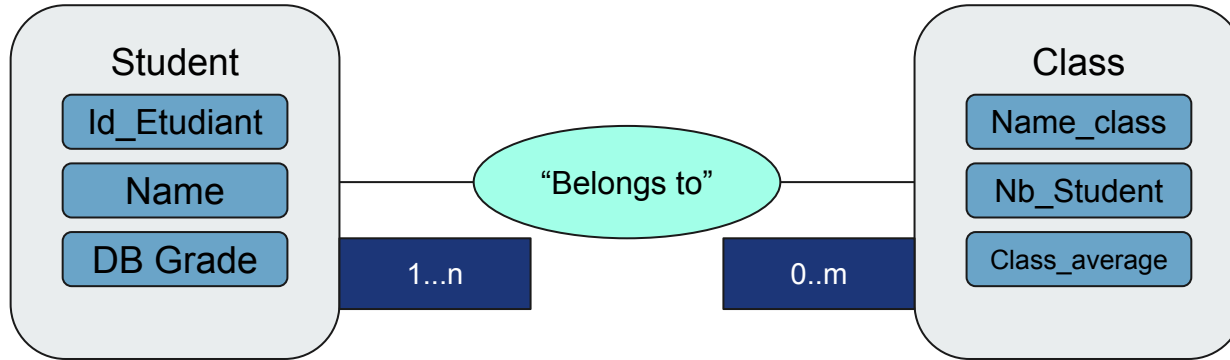
Association Entity Concept



Example of Entity-Relational Model (ER Model)

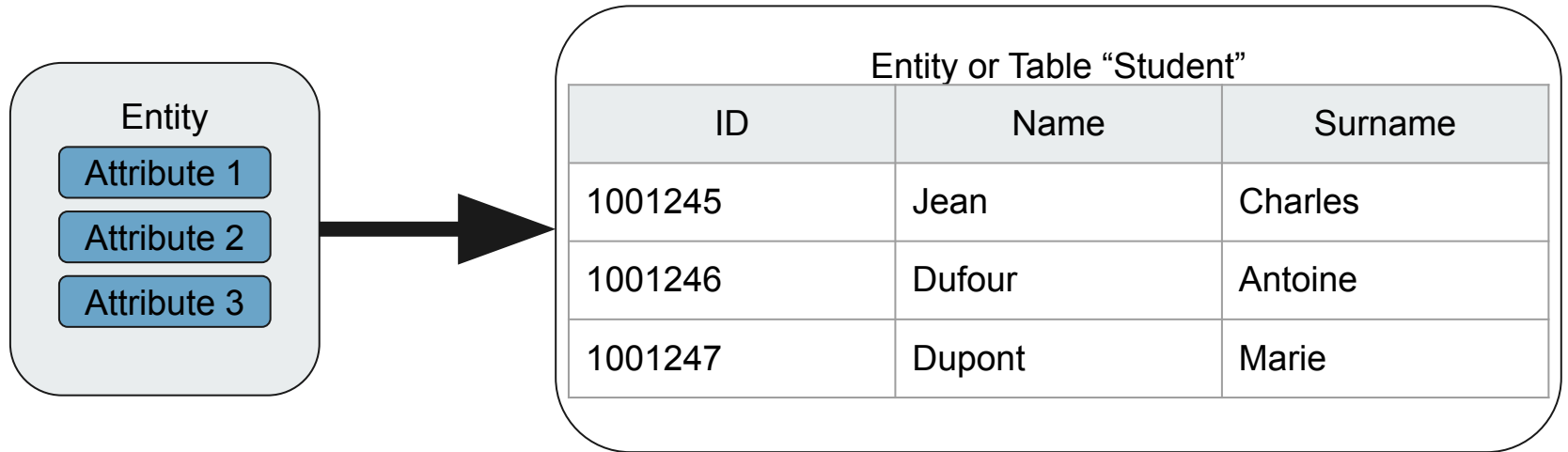
Association Entity Concept

Let's try to make this example concrete...



Association Entity Concept

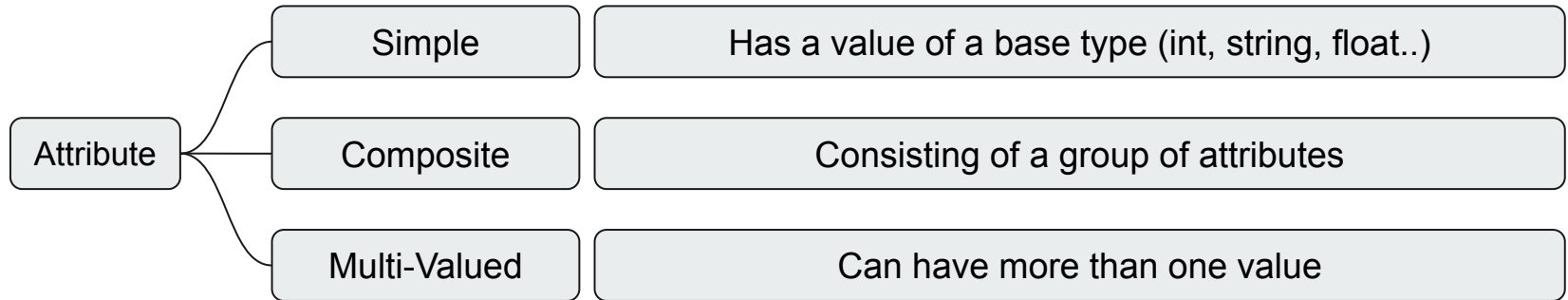
The entity represents an abstract object that can contain a set of attributes dedicated to it.





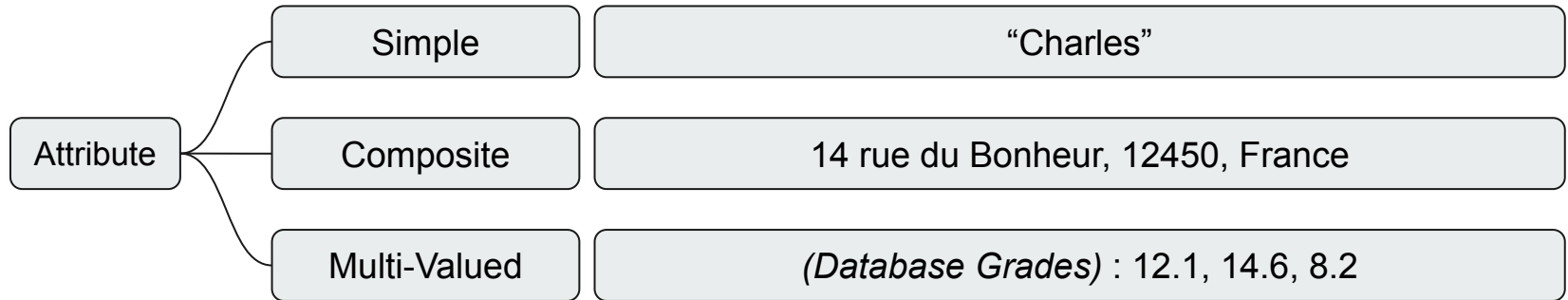
Attributes in the Entity Association Model

Attributes can be of different forms:



Attributes in the Entity Association Model

Attributes can be of different forms:





Association Entity Concept

The association is what will allow to make the link between 2 databases in the Entity Association model. It takes the form of an active verb.

“Belongs to”, “possesses”, “passes” etc.





Association Entity Concept - Cardinality

Related to the association, cardinality represents a pair of values that imposes a constraint on the Entity/Association model.

The possible cardinalities are:

1...n

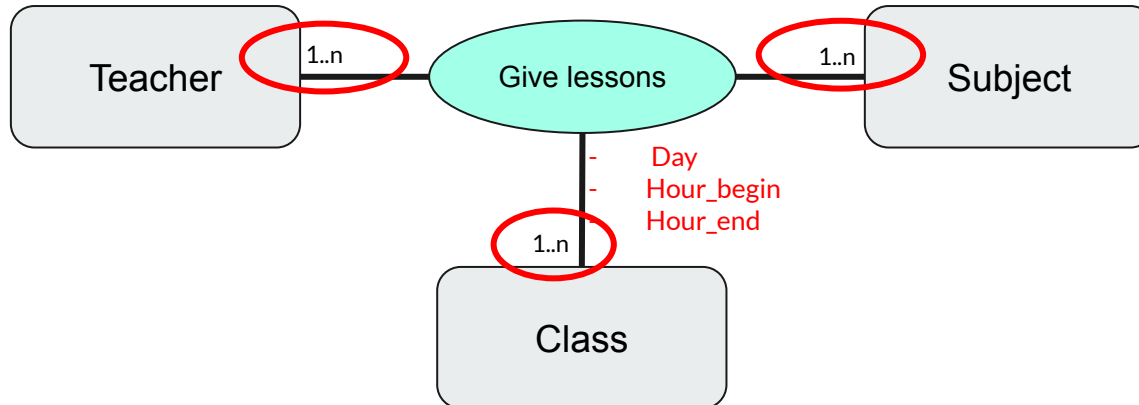
0..1

0...n

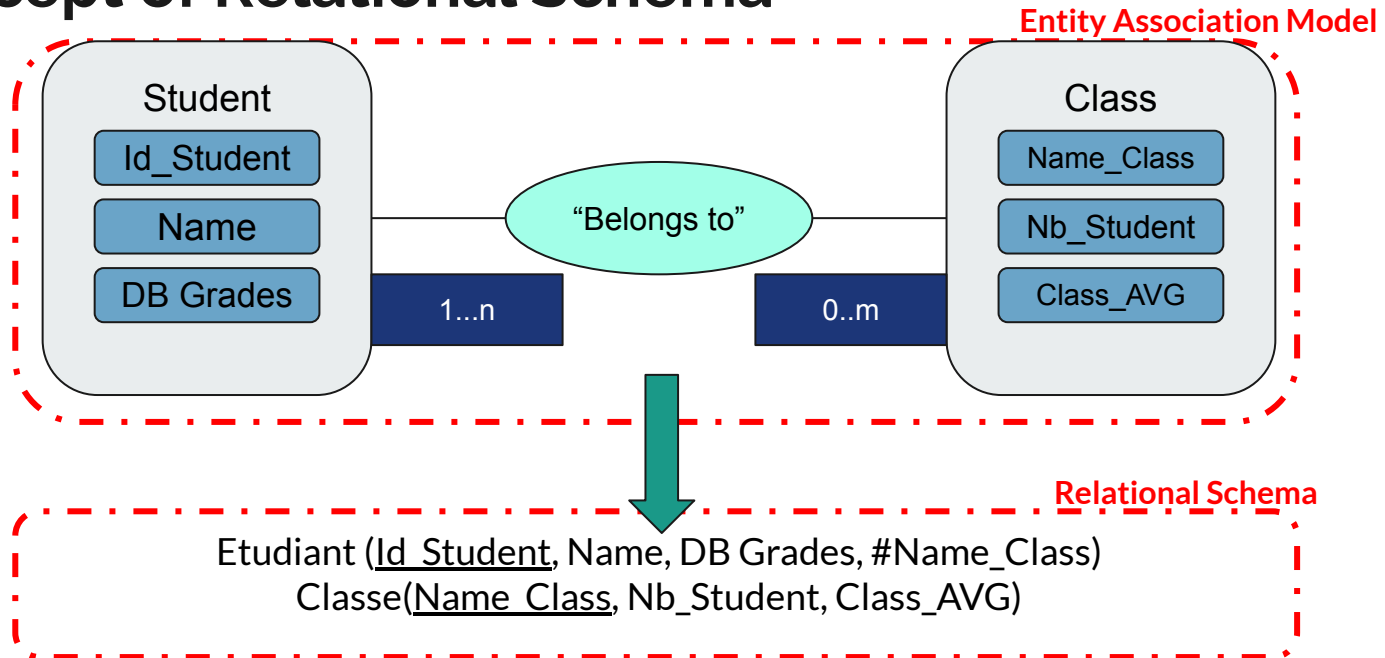
1..1

N-ary association

If the cardinality of each entity is the same on each side of the association, the association can be assigned an entity.



Concept of Relational Schema



Concept of Relational Schema

The concept of Relational Schema is closer to the computer design of databases :

Etudiant (Id_Student, Name, DB Grades, #Name_Class)
Classe(Name_Class, Nb_Student, Class_AVG)



Student Table

Id_Student	Name	DB Grade	Name_Class
1001245	Jean	12.5	3A
1002213	Patrick	14.1	2A

Class Table

Name_Class	Nb_Student	Class_AVG
3A	35	12.76
2A	31	13.31

Primary keys (Identifiers) and foreign keys

A primary key (also called an “identifier”) is an attribute that allows you to find a unique instance among all those in the table.

An identifier can be made up of several attributes.

Student (Id_Student, Name, DB_Grades, #Name_class)



Primary key

Maison (N°, Street, City, size, price)



Composite Primary Key

Primary keys (Identifiers) and foreign keys

Foreign keys are primary keys that do not belong natively to the entity that owns them.

It allows you to make a join between 2 tables.

Etudiant (ID_Student, Name, DB Grade, #Name_class)
Classe(Name_class, Nb_Students, Class_AVG)



Id_Student	Name	DB Grade	Name_class
1001245	Jean	12.5	3A
1002213	Patrick	14.1	2A

Name_class	Nb_Students	Class_AVG
3A	35	12.76
2A	31	13.31



Primary keys (Identifiers) and foreign keys

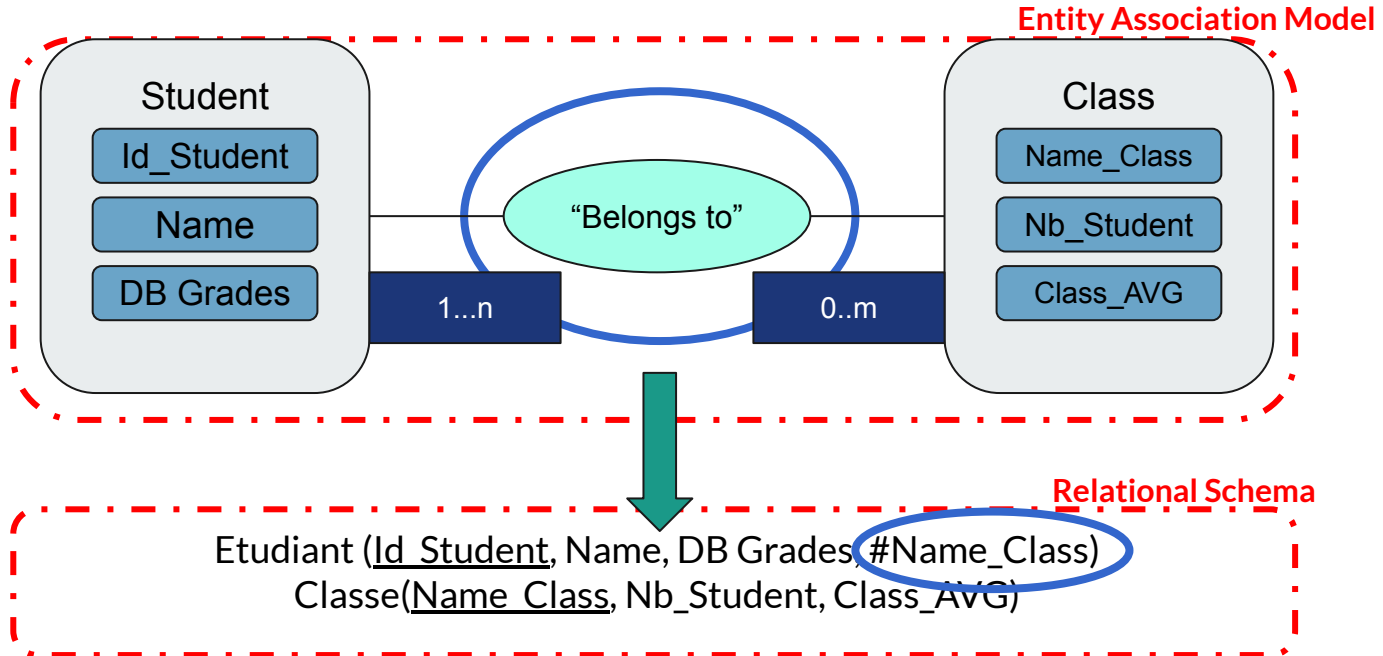
Etudiant (ID_Student, Name, DB Grade, #Name_class)
Classe(Name_class, Nb_Students, Class_AVG)

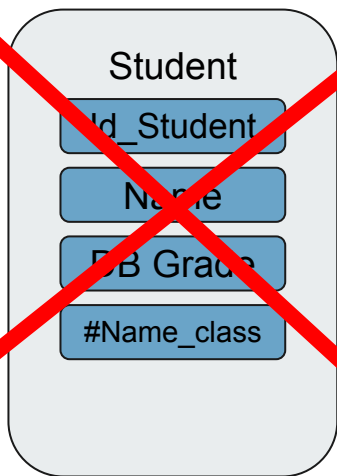
Here, “Class_Name” is the primary key of the “Class” entity but is the foreign key of the “Student” entity



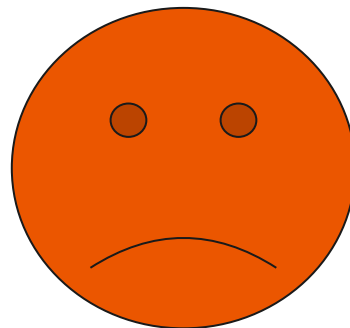
The Entity Association model CANNOT CONTAIN FOREIGN keys







0/20





Classic examination statement

[Descriptive text of the situation]

Q1: “Give the Entity Association model corresponding to the description in the text above.”

Q2: “Deduce the associated relational schema”



Application exercise

Statement: A customer has a social security number, a name, a first name, a bonus and a penalty

He signs a contract with an agent for each of his vehicles, for certain risks covered. A vehicle is characterized by a number, a power, a brand, a type and a color. Customers sometimes have claims with third parties. Third parties have a social security number and are insured with an insurance company. An insurance company has a name and an address. We must know the place and date of the claim.

Q1: "Give the Entity Association model corresponding to the description of the text above."

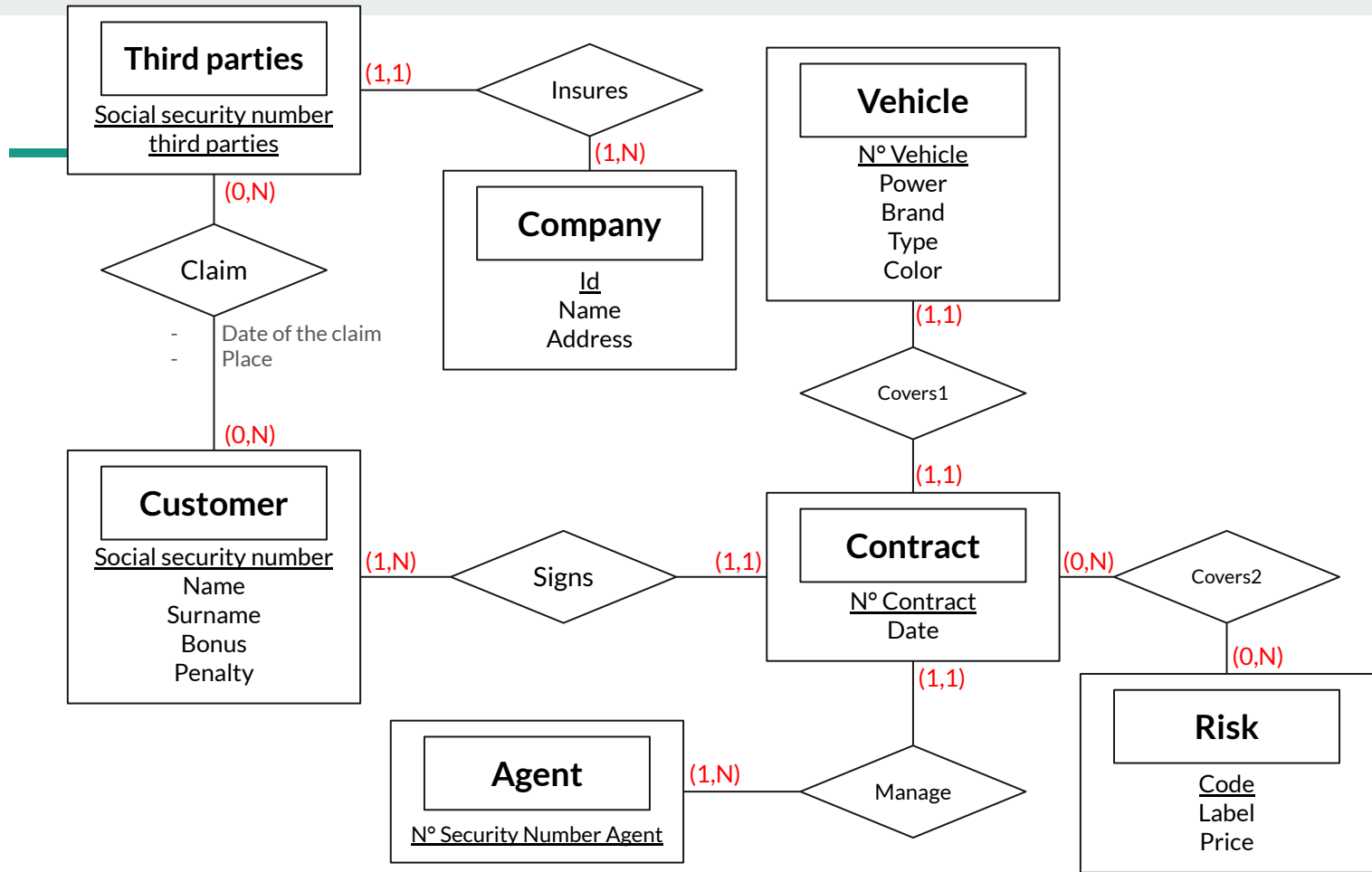
Q2: "Deduce the associated relational schema"



Application exercise

Statement: A **customer** has a **social security number**, a **name**, a **first name**, a **bonus** and a **penalty**

He **signs** a **contract** with an **agent** for each of his **vehicles**, for certain **risks covered**. A vehicle is characterized by a **number**, a **power**, a **brand**, a **type** and a **color**. Customers sometimes have **claims** with **third parties**. Third parties have a **social security number** and **are insured** with an **insurance company**. An **insurance company** has a **name** and an **address**. We must know the **place** and **date of the claim**.



Associated Relational Schema

Customer (Social security number, Name, Surname, Bonus, Penalty)

Contract (N° Contract, Date, #Social security number, #Number, #N° Security Number Agent)

Agent (N° Security Number Agent, Name_Agent, Surname_Agent, Adress_Agent)

Vehicle (N° Vehicle, Power, Brand, Color, Type, #N°Contract)

Risk (Code, Label, Price)

Third Parties (Social security number third parties,..., #Id_Company)

Company (Id_Company, Name_company, Adress_company)

Claim (N°Claim, Date_Claim, place, #Social security number, #Social security number third parties)

Covers2 (N°Covers, N° Contract, Code)



For a particular information system, there is no single conceptual model. The right model is the one validated by all members of the project.



Course 2

Tutorial 1

Entity Association Model Exercises

Relational Diagram



Tutorial 1

Go to file “Tutorial Relational Databases.pdf” and begin section 1.



Cours 3

Presentation of Relational and non-relational Databases

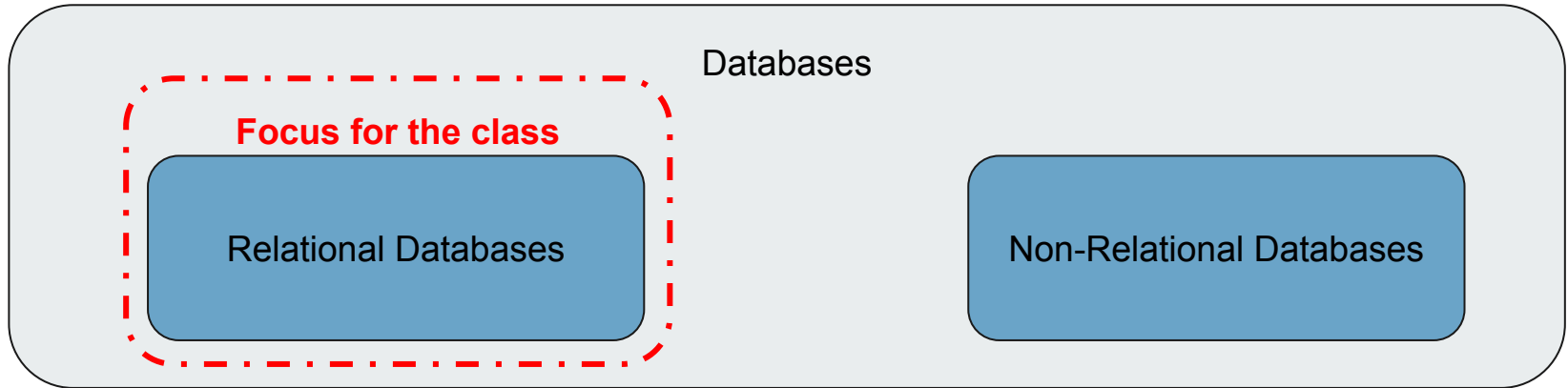
WampServer software Installation

Presentation of a SQL query - decomposition



Présentation des bases de données

There are two main types of databases: **Relational** and **Non-Relational**.



Technologies linked to relational databases



Relational Databases





MySQL

MySQL is a open-source RDBMS (Relational database management system), developed by Oracle.

MySQL  SQL

Don't confuse MySQL (RDBMS) with SQL (Structured Query Language) which is a query language to make some requests on relational databases.



MySQL and MariaDB

MySQL and MariaDB are software that allow us to exploit a database. They serve as an interface between the user and the database.





Installation SQLite

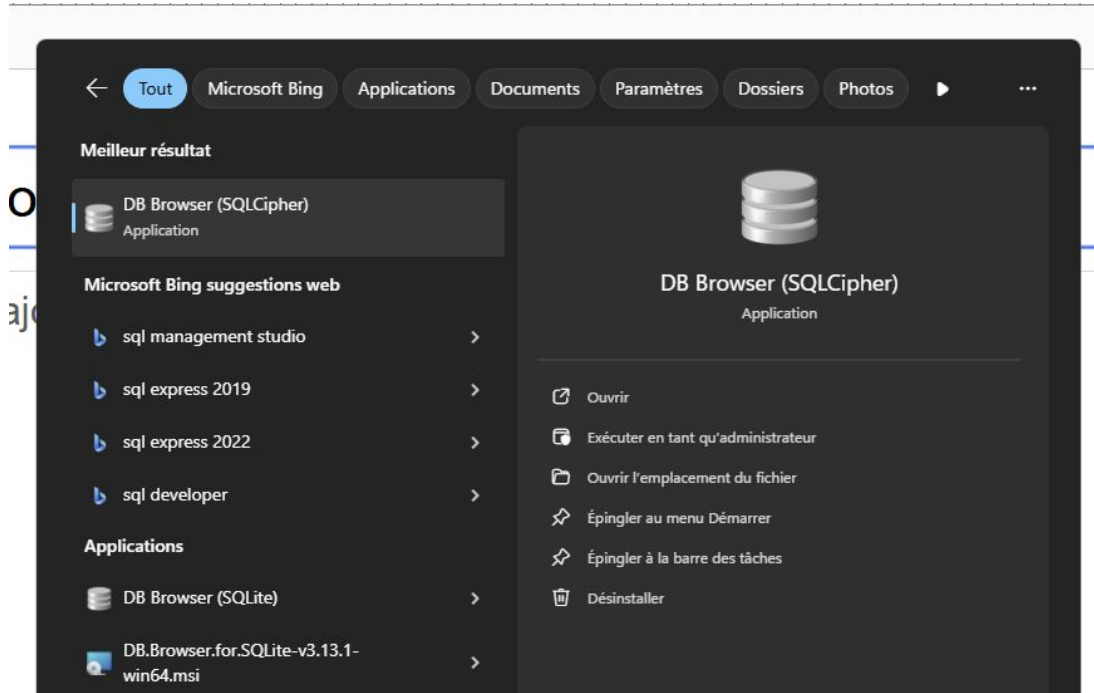
<https://www.sqlite.org/>

Take the launcher

Download and use the launcher

Nom	Modifié le	Type	Taille
✓ Aujourd'hui			
 script1-db	20/01/2026 13:03	Fichier SQL	2 Ko
 MACOSX	20/01/2026 12:58	Dossier de fichiers	
 WINDOWS	20/01/2026 12:58	Dossier de fichiers	

Then, open the software



Click on button “Open a database”

DB Browser for SQLite

Fichier Édition Vue Outils Aide

Nouvelle base de données

Ouvrir une base de données

Enregistrer les modifications

Annuler les modifications

Annuler

Ouvrir un projet

Enregistrer le projet

Structure de la base de données

Récupérer les données

Éditer les pragmas

Exécuter le SQL

Créer une table

Créer un index

Modifier la table

Supprimer la table

Imprimer

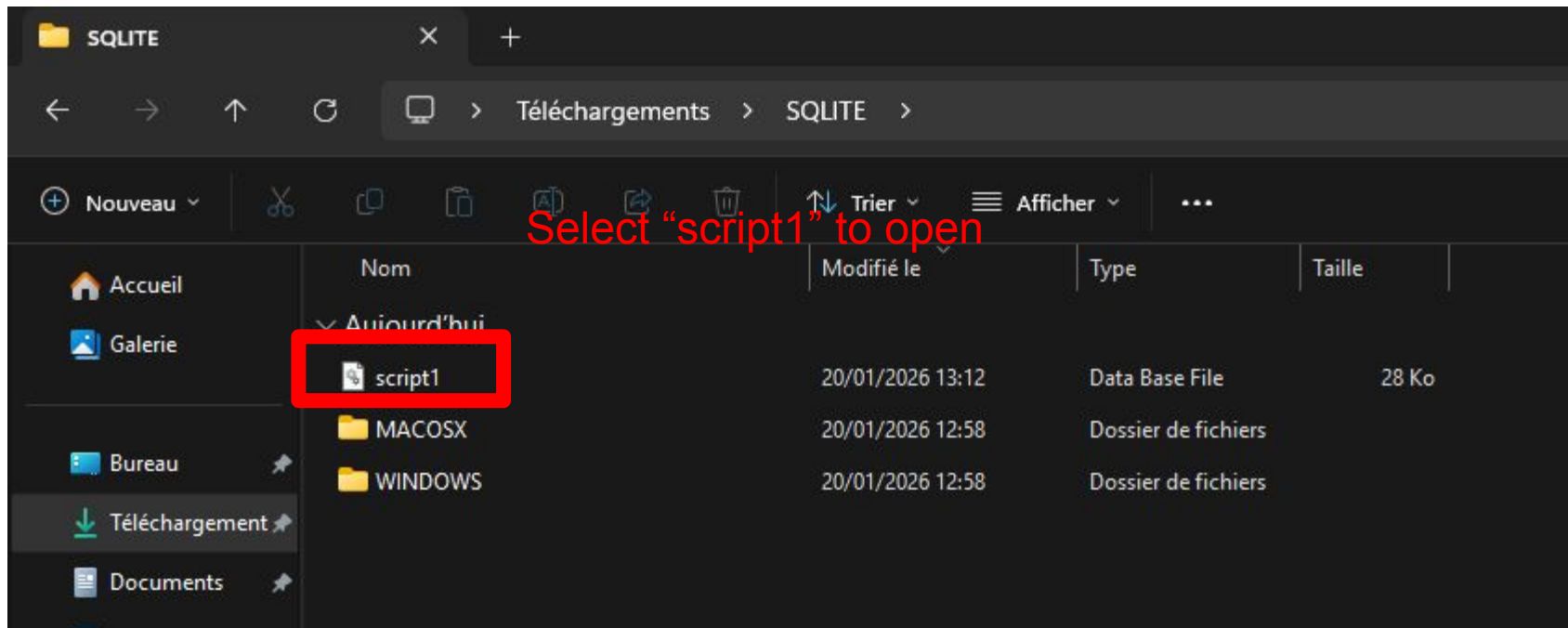
Rafraîchir

Nom

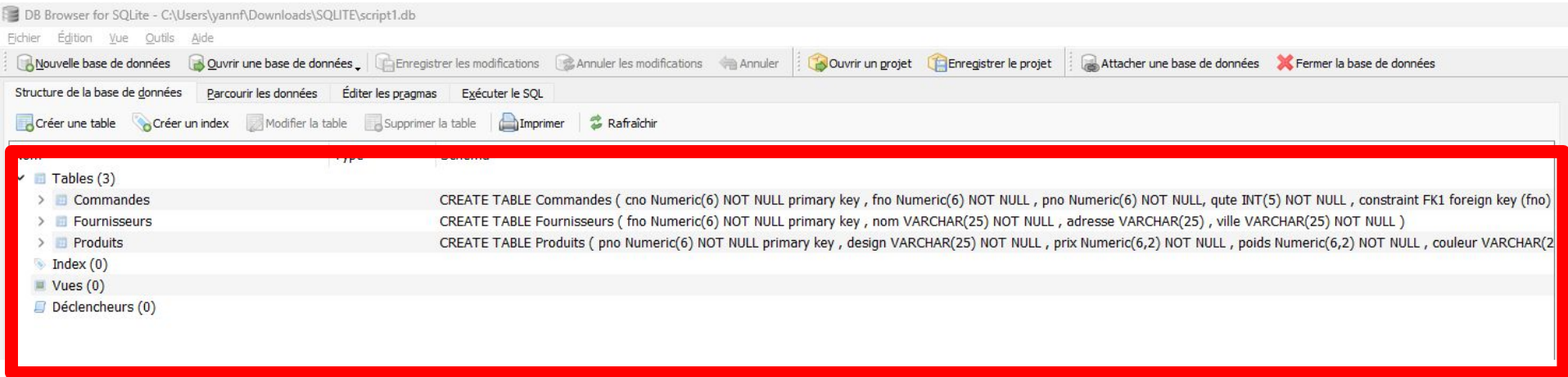
Type

Schéma

Selection du script



You just created your first database

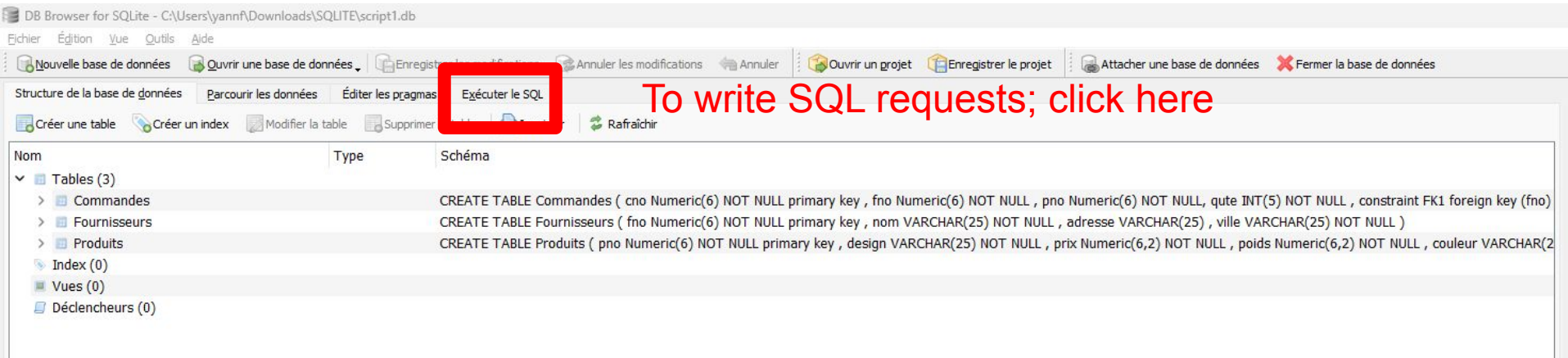


The screenshot shows the DB Browser for SQLite application. The title bar indicates the file path: C:\Users\yannf\Downloads\SQLITE\script1.db. The menu bar includes 'Fichier', 'Édition', 'Vue', 'Outils', and 'Aide'. The toolbar contains buttons for 'Nouvelle base de données', 'Ouvrir une base de données', 'Enregistrer les modifications', 'Annuler les modifications', 'Annuler', 'Ouvrir un projet', 'Enregistrer le projet', 'Attacher une base de données', and 'Fermer la base de données'. Below the toolbar, there are tabs for 'Structure de la base de données', 'Parcourir les données', 'Éditer les pragmas', and 'Exécuter le SQL'. The 'Structure de la base de données' tab is active, showing a tree view of the database structure. The tree view is highlighted with a red border. It shows three tables: 'Commandes', 'Fournisseurs', and 'Produits'. Each table has a corresponding 'CREATE TABLE' SQL statement displayed next to it. The 'Index (0)' and 'Vues (0)' sections are also visible, indicating no indexes or views are present. The 'Déclencheurs (0)' section is also visible, indicating no triggers are present.

nom	type	contenu
✓ Tables (3)		
> Commandes		CREATE TABLE Commandes (cno Numeric(6) NOT NULL primary key , fno Numeric(6) NOT NULL , pno Numeric(6) NOT NULL, qute INT(5) NOT NULL , constraint FK1 foreign key (fno)
> Fournisseurs		CREATE TABLE Fournisseurs (fno Numeric(6) NOT NULL primary key , nom VARCHAR(25) NOT NULL , adresse VARCHAR(25) , ville VARCHAR(25) NOT NULL)
> Produits		CREATE TABLE Produits (pno Numeric(6) NOT NULL primary key , design VARCHAR(25) NOT NULL , prix Numeric(6,2) NOT NULL , poids Numeric(6,2) NOT NULL , couleur VARCHAR(2
Index (0)		
Vues (0)		
Déclencheurs (0)		

The Content of the table is here

You just created your first database



The screenshot shows the DB Browser for SQLite application. The title bar indicates the file path: C:\Users\yannf\Downloads\SQLITE\script1.db. The menu bar includes 'Fichier', 'Édition', 'Vue', 'Outils', and 'Aide'. The toolbar contains buttons for 'Nouvelle base de données', 'Ouvrir une base de données', 'Enregistrer le projet', 'Annuler les modifications', 'Annuler', 'Ouvrir un projet', 'Enregistrer le projet', 'Attacher une base de données', and 'Fermer la base de données'. Below the toolbar, there are buttons for 'Structure de la base de données', 'Parcourir les données', 'Éditer les pragmas', and 'Exécuter le SQL'. The 'Exécuter le SQL' button is highlighted with a red rectangle. To the right of this button, red text reads 'To write SQL requests; click here'. Below the toolbar, there are buttons for 'Créer une table', 'Créer un index', 'Modifier la table', 'Supprimer', and 'Rafraîchir'. The main area displays a table with three columns: 'Nom', 'Type', and 'Schéma'. The table lists three tables: 'Commandes', 'Fournisseurs', and 'Produits', each with its corresponding CREATE TABLE SQL statement. Below the table, there are sections for 'Index (0)', 'Vues (0)', and 'Déclencheurs (0)'.

DB Browser for SQLite - C:\Users\yannf\Downloads\SQLITE\script1.db

Fichier Édition Vue Outils Aide

Nouvelle base de données Ouvrir une base de données Enregistrer le projet Annuler les modifications Annuler Ouvrir un projet Enregistrer le projet Attacher une base de données Fermer la base de données

Structure de la base de données Parcourir les données Éditer les pragmas Exécuter le SQL

Créer une table Créer un index Modifier la table Supprimer Rafraîchir

Nom	Type	Schéma
Tables (3)		
Commandes		CREATE TABLE Commandes (cno Numeric(6) NOT NULL primary key , fno Numeric(6) NOT NULL , pno Numeric(6) NOT NULL , qute INT(5) NOT NULL , constraint FK1 foreign key (fno)
Fournisseurs		CREATE TABLE Fournisseurs (fno Numeric(6) NOT NULL primary key , nom VARCHAR(25) NOT NULL , adresse VARCHAR(25) , ville VARCHAR(25) NOT NULL)
Produits		CREATE TABLE Produits (pno Numeric(6) NOT NULL primary key , design VARCHAR(25) NOT NULL , prix Numeric(6,2) NOT NULL , poids Numeric(6,2) NOT NULL , couleur VARCHAR(2
Index (0)		
Vues (0)		
Déclencheurs (0)		

DB Browser for SQLite - C:\Users\yannf\Downloads\SQLite\script1.db

Fichier Édition Vue Outils Aide

Nouvelle base de données Ouvrir une base de données Enregistrer les modifications Annuler les modifications Annuler Ouvrir un projet Enreg

Structure de la base de données Parcourir les données Éditer les pragmas Exécuter le SQL

1 SELECT * FROM Commandes

Write your SQL Requests here

	cno	fno	pno	qute
1	1001	17	103	10
2	1003	15	103	2
3	1005	17	102	1
4	1007	15	108	1
5	1011	19	107	12
6	1013	13	107	5
7	1017	19	105	3

L'exécution s'est terminée sans erreur.
Résultat : 10 enregistrements ramenés en 18ms
À la ligne 1 :
SELECT * FROM Commandes

DB Browser for SQLite - C:\Users\yannf\Downloads\SQLITE\script1.db

Fichier Édition Vue Outils Aide

Nouvelle base de données Ouvrir une base de données Enregistrer les modifications Annuler les modifications Annuler Ouvrir un projet Enreg

Structure de la base de données Parcourir les données Éditer les pragmas Exécuter le SQL

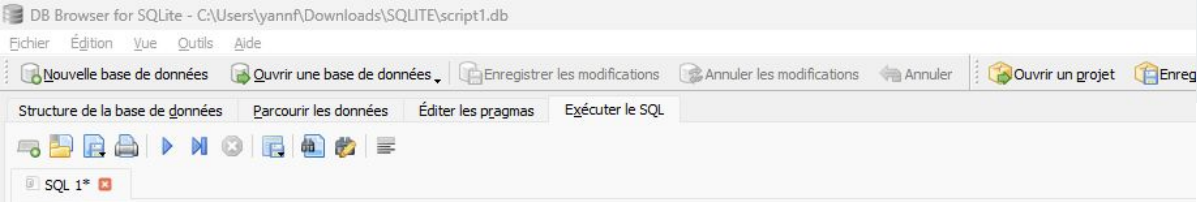
SQL 1*

```
1 SELECT * FROM Commandes
```

Execute your SQL code here

	cno	fno	pno	qute
1	1001	17	103	10
2	1003	15	103	2
3	1005	17	102	1
4	1007	15	108	1
5	1011	19	107	12
6	1013	13	107	5
7	1017	19	105	3

L'exécution s'est terminée sans erreur.
Résultat : 10 enregistrements ramenés en 18ms
À la ligne 1 :
SELECT * FROM Commandes



Result of your SQL Request going to be here

cno	fno	pno	qute
1001	17	103	10
1003	15	103	2
1005	17	102	1
1007	15	108	1
1011	19	107	12
1013	13	107	5
1017	19	105	3

'exécution s'est terminée sans erreur.
résultat : 10 enregistrements ramenés en 18ms
la ligne 1 :
SELECT * FROM Commandes



Presentation of a SQL query

A SQL query for data manipulation is written by this way :

[Action] [Element] FROM [Table] WHERE [Condition]



Exemples

“SELECT * FROM commandes” - Select all the entries in the table commandes

“SELECT * FROM commandes WHERE fno = 15” - Select all the entries in the table commandes where the attribute “fno” is equal to 15.



A utilisier

https://www.w3schools.com/sql/sql_syntax.asp



Cours 6

Data manipulation



SELECT

The **SELECT** statement is used to select data from a database.

```
SELECT nom_du_champ FROM nom_du_tableau
```

***SELECT** * FROM commandes*



DISTINCT

The **SELECT DISTINCT** statement is used to return only distinct (different) values.

“Sélectionner different fno in the table commandes”

```
SELECT DISTINCT fno FROM commandes
```



WHERE

The **WHERE** clause is used to filter records.

It is used to extract only those records that fulfill a specified condition.

“Sélectionner les commandes dont le pno est supérieur à 104”

```
SELECT * FROM commandes WHERE pno > 104
```



AND & OR

The **AND** operator displays a record if *all* the conditions are TRUE.

The **OR** operator displays a record if *any* of the conditions are TRUE.

The **WHERE** clause can contain one or many **AND** operators.

```
SELECT * FROM commandes WHERE fno>15 OR fno<12
```

```
SELECT * FROM commandes WHERE fno = 15 AND pno =12
```



IN

The **IN** operator allows you to specify multiple values in a **WHERE** clause.

The **IN** operator is a shorthand for multiple **OR** conditions.



BETWEEN

The **BETWEEN** operator selects values within a given range. The values can be numbers, text, or dates.

The **BETWEEN** operator is inclusive: begin and end values are included.

```
SELECT * FROM commandes WHERE pno > 12 AND pno < 15
```

```
⇔ SELECT * FROM commandes WHERE pno BETWEEN 12 AND 15
```



LIKE

The **LIKE** operator is used in a **WHERE** clause to search for a specified pattern in a column.

There are two wildcards often used in conjunction with the **LIKE** operator:

- The percent sign **%** represents zero, one, or multiple characters
- The underscore sign **_** represents one, single character
-



IS NULL / IS NOT NULL

L'opérateur IS permet de filtrer les résultats qui contiennent la valeur NULL. Cet opérateur est indispensable car la valeur NULL est une valeur inconnue et ne peut par conséquent pas être filtrée par les opérateurs de comparaison (cf. égal, inférieur, supérieur ou différent).



Cours 7

Les Créations et modifications de table



CREATE DATABASE / CREATE TABLE

La création d'une base de données en SQL est possible en ligne de commande grâce à la commande CREATE DATABASE. Même si les systèmes de gestion de base de données (SGBD) sont souvent utilisés pour créer une base, il convient de connaître la commande à utiliser, qui est très simple. Pour créer une table dans une base de données, on utilise CREATE TABLE.

```
CREATE DATABASE IF NOT EXISTS esgf_db
```

```
CREATE TABLE IF NOT EXISTS commandes
```



ALTER TABLE

La commande ALTER TABLE permet de modifier une table existante. Idéal pour ajouter une colonne, supprimer une colonne ou modifier une colonne existante, par exemple pour changer le type.

ALTER TABLE commandes

ADD nom_commande CHAR

```
ALTER TABLE nom_table
```

```
ADD nom_colonne type_donnees
```



UPDATE

La commande UPDATE permet d'effectuer des modifications sur des lignes existantes. Très souvent cette commande est utilisée avec WHERE pour spécifier sur quelles lignes doivent porter la ou les modifications.

```
UPDATE table
```

```
SET nom_colonne_1 = 'nouvelle valeur'
```

```
WHERE condition
```



DELETE

La commande DELETE en SQL permet de supprimer des lignes dans une table. En utilisant cette commande associé à WHERE il est possible de sélectionner les lignes concernées qui seront supprimées.

```
DELETE FROM `table`
```

```
WHERE condition
```



TRUNCATE TABLE

En SQL, la commande **TRUNCATE TABLE** permet de supprimer toutes les données d'une table sans supprimer la table en elle-même. En d'autres mots, cela permet de purger la table. Cette instruction diffère de la commande **DROP** qui à pour but de supprimer les données ainsi que la table qui les contient.



GROUP BY

La commande GROUP BY est utilisée en SQL pour grouper plusieurs résultats et utiliser une fonction de totaux sur un groupe de résultat. Sur une table qui contient toutes les ventes d'un magasin, il est par exemple possible de liste regrouper les ventes par clients identiques et d'obtenir le coût total des achats pour chaque client.

```
SELECT colonne1, fonction(colonne2)
```

```
FROM table
```

```
GROUP BY colonne1
```



ORDER BY

La commande ORDER BY permet de trier les lignes dans un résultat d'une requête SQL. Il est possible de trier les données sur une ou plusieurs colonnes, par ordre ascendant ou descendant.

```
SELECT colonne1, colonne2
```

```
FROM table
```

```
ORDER BY colonne1
```



AS

Dans le langage SQL il est possible d'utiliser des **alias** pour renommer temporairement une colonne ou une table dans une requête. Cette astuce est particulièrement utile pour faciliter la lecture des requêtes.

```
SELECT * FROM commandes AS com WHERE com.pno = 12
```



Cours 8

SQL Queries



TD/TP3

Rendez vous sur le fichier “TD TP BDDR.pdf” et commencez la section 3 - SQL1



Cours 9

SQL Queries



TD/TP3

Rendez vous sur le fichier “TD TP BDDR.pdf” et continuez la section 3 - SQL1



Cours 10

Les fonctions d'agrégation



HAVING

La condition HAVING en SQL est presque similaire à WHERE à la seule différence que HAVING permet de filtrer en utilisant des fonctions telles que SUM(), COUNT(), AVG(), MIN() ou MAX().

```
SELECT colonne1, SUM(colonne2)
```

```
FROM nom_table
```

```
GROUP BY colonne1
```

```
    HAVING fonction(colonne2) operateur valeur
```



AVG

La fonction d'agrégation `AVG()` dans le langage SQL permet de calculer une valeur moyenne sur un ensemble d'enregistrement de type numérique et non nul.



COUNT

En SQL, la fonction d'agrégation COUNT() permet de compter le nombre d'enregistrement dans une table. Connaître le nombre de lignes dans une table est très pratique dans de nombreux cas, par exemple pour savoir combien d'utilisateurs sont présents dans une table ou pour connaître le nombre de commentaires sur un article.



MAX

Dans le langage SQL, la fonction d'agrégation MAX() permet de retourner la valeur maximale d'une colonne dans un set d'enregistrement. La fonction peut s'appliquée à des données numériques ou alphanumériques. Il est par exemple possible de rechercher le produit le plus cher dans une table d'une boutique en ligne.



MIN

La fonction d'agrégation MIN() de SQL permet de retourner la plus petite valeur d'une colonne sélectionnée. Cette fonction s'applique aussi bien à des données numériques qu'à des données alphanumériques.



SUM

Dans le langage SQL, la fonction d'agrégation SUM() permet de calculer la somme totale d'une colonne contenant des valeurs numériques. Cette fonction ne fonctionne que sur des colonnes de types numériques (INT, FLOAT ...) et n'additionne pas les valeurs NULL.



Cours 11

SQL Queries



TD/TP3

Rendez vous sur le fichier “TD TP BDDR.pdf” et commencez la section 3 - SQL2



Cours 12

Évaluation sur table



Cours 13

Jointures



INNER JOIN

Dans le langage SQL la commande INNER JOIN, aussi appelée EQUIJOIN, est un type de jointures très communes pour lier plusieurs tables entre-elles. Cette commande retourne les enregistrements lorsqu'il y a au moins une ligne dans chaque colonne qui correspond à la condition.

```
SELECT *
```

```
FROM table1
```

```
INNER JOIN table2 ON table1.id = table2.fk_id
```



CROSS JOIN

Dans le langage SQL, la commande CROSS JOIN est un type de jointure sur 2 tables SQL qui permet de retourner le produit cartésien. Autrement dit, cela permet de retourner chaque ligne d'une table avec chaque ligne d'une autre table. Ainsi effectuer le produit cartésien d'une table A qui contient 30 résultats avec une table B de 40 résultats va produire 1200 résultats ($30 \times 40 = 1200$). En général la commande CROSS JOIN est combinée avec la commande WHERE pour filtrer les résultats qui respectent certaines conditions.

```
SELECT *  
  
FROM table1  
  
    CROSS JOIN table2
```



LEFT JOIN

Dans le langage SQL, la commande LEFT JOIN (aussi appelée LEFT OUTER JOIN) est un type de jointure entre 2 tables. Cela permet de lister tous les résultats de la table de gauche (left = gauche) même s'il n'y a pas de correspondance dans la deuxième tables.

```
SELECT *
```

```
FROM table1
```

```
LEFT JOIN table2 ON table1.id = table2.fk_id
```



RIGHT JOIN

En SQL, la commande RIGHT JOIN (ou RIGHT OUTER JOIN) est un type de jointure entre 2 tables qui permet de retourner tous les enregistrements de la table de droite (right = droite) même s'il n'y a pas de correspondance avec la table de gauche. S'il y a un enregistrement de la table de droite qui ne trouve pas de correspondance dans la table de gauche, alors les colonnes de la table de gauche auront NULL pour valeur.

```
SELECT *
```

```
FROM table1
```

```
RIGHT JOIN table2 ON table1.id = table2.fk_id
```



FULL JOIN

Dans le langage SQL, la commande FULL JOIN (ou FULL OUTER JOIN) permet de faire une jointure entre 2 tables. L'utilisation de cette commande permet de combiner les résultats des 2 tables, les associer entre eux grâce à une condition et remplir avec des valeurs NULL si la condition n'est pas respectée.

```
SELECT *  
  
FROM table1  
  
FULL JOIN table2 ON table1.id = table2.fk_id
```



SELF JOIN

En SQL, un SELF JOIN correspond à une jointure d'une table avec elle-même. Ce type de requête n'est pas si commun mais très pratique dans le cas où une table lie des informations avec des enregistrements de la même table.

```
SELECT `t1`.`nom_colonne1`, `t1`.`nom_colonne2`, `t2`.`nom_colonne1`, `t2`.`nom_colonne2`  
  
FROM `table` as `t1`  
  
LEFT OUTER JOIN `table` as `t2` ON `t2`.`fk_id` = `t1`.`id`
```



NATURAL JOIN

Dans le langage SQL, la commande NATURAL JOIN permet de faire une jointure naturelle entre 2 tables. Cette jointure s'effectue à la condition qu'il y ai des colonnes du même nom et de même type dans les 2 tables. Le résultat d'une jointure naturelle est la création d'un tableau avec autant de lignes qu'il y a de paires correspondant à l'association des colonnes de même nom.

```
SELECT *
```

```
FROM table1
```

```
NATURAL JOIN table2
```



Course 14

SQL Queries



TD/TP3

Rendez vous sur le fichier “TD TP BDDR.pdf” et continuez la section 3 - SQL2 - Jointures.